

Movie Explorer — Bug Book

Bugs I hit while building infinite scroll, search and filters for the movie app — what I saw, why it happened, and how I fixed it, in plain words.

React 19 · TypeScript · Vite · TMDB API · IntersectionObserver

Bugs in this book

- | | |
|--|--|
| #4 Pages load by themselves on an empty screen | #13 Search and popular movies load pages together |
| #5 Fast scrolling loads too many pages | #14 A new search can't load more pages |
| #6 "No Movies Available" flashes before the movies appear | #15 New page wipes out the old one |
| #10 Movie details page is completely blank | #16 Search loads once, then stops |
| #11 Two useFetch calls use the same names | #17 Observer keeps restarting on every render |
| | #20 Loading stops when filters are on |

Words used in this book

Sentinel	An invisible 10px div at the bottom of the movie list. When it shows up on screen, we load the next page.
Observer	The IntersectionObserver. It watches the sentinel and tells us when it goes from hidden to visible, or visible to hidden.
Screen (viewport)	The part of the page you can see right now.
Ref	A box from useRef that always holds the latest value. Changing it does not re-render.
State	A value from useState. Changing it re-renders, but each render only sees its own copy.

The basics

How the sentinel and observer work

In the DOM vs on screen

In the DOM

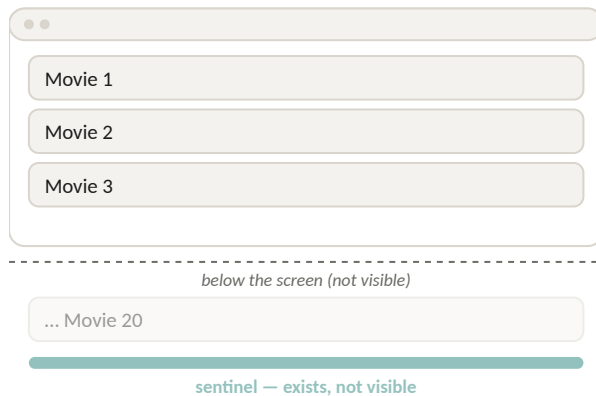
The element exists on the page. `{popularMovies.length > 0 && <div ref={sentinalRef} />}` only decides this. It could still be far down, where you can't see it.

On screen

The element is inside the part of the page you can currently see.

Think of the page as a long strip of paper and your screen as a window that shows one part of it. Scrolling moves the window. "Pushed below the screen" means new movies were added above the sentinel, so it moved further down, past the bottom edge of the window.

In the DOM, but not on screen



Scrolled down: now on screen



How the observer knows

You don't calculate anything. Whenever the page is drawn (you scroll, or new movies appear), the browser checks whether the sentinel's box overlaps your screen's box. If yes, it's visible. When that answer changes, the browser calls the observer with `isIntersecting = true` or `false`.

The rule behind most of these bugs: The observer only speaks up when visibility changes: hidden → visible, or visible → hidden. If the sentinel stays visible, it stays quiet. When an observer first starts watching, it also checks once right away.

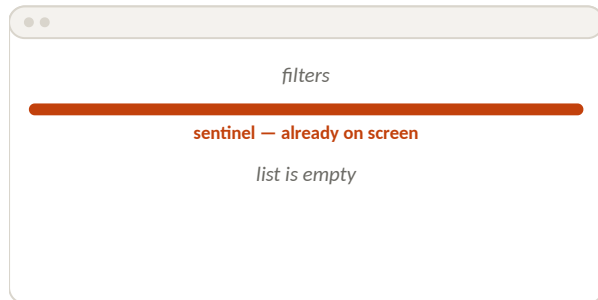
Bug #4

Pages load by themselves on an empty screen

What you saw

On a fresh page load, pages 2, 3, 4 and more were requested before a single movie appeared — without touching the scroll wheel.

● Before — sentinel always on the page



Observer sees it right away → page 2, 3, 4...

● After — sentinel only shows when there are movies



Why it happened

When the page first opens, the movie list is empty. With nothing above it, the sentinel sits near the top of the screen — so it's already visible.

When the observer starts watching something, it checks right away: "is it visible right now?" It was, so it asked for the next page.

The list was still empty while that page loaded, so the sentinel was still visible, so it asked again, and again.

In one line: empty list → sentinel sits at the top → observer sees it right away → pages load by themselves.

The code

Before — sentinel always on the page

```
<MovieList movies={movies} />
// always on the page, even when the list is empty
<div ref={sentinelRef} style={{ height: 10 }} />
```

After — sentinel only shows when there are movies

```
<MovieList movies={movies} />
// only add it once there are movies to push it down
{movies.length > 0 && (
  <div ref={sentinelRef} style={{ height: 10 }} />
)}
```

How to avoid it next time

- Don't show the sentinel until the list has at least one item.

- Always check `isFetchingRef.current` before loading more, as a backup.
- Remember: the observer checks right away when it starts watching — it doesn't wait for a scroll.

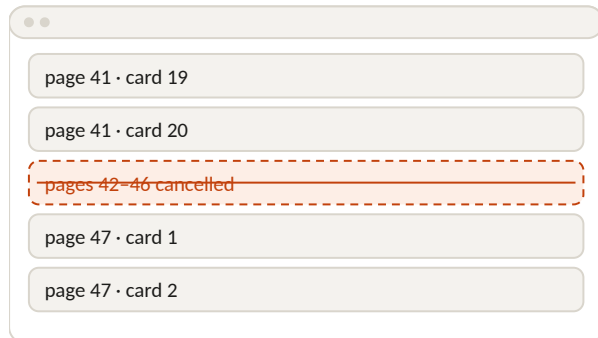
Bug #5

Fast scrolling loads too many pages

What you saw

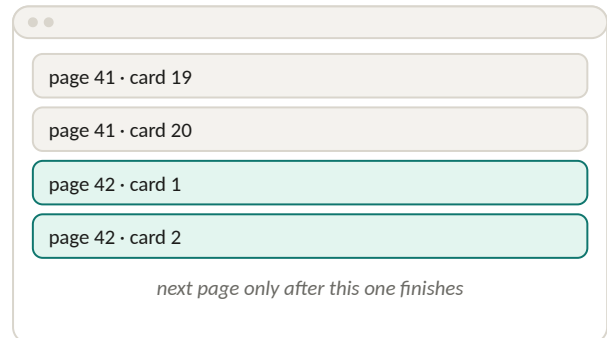
Scrolling slowly worked fine. A fast flick fired many requests at once — the page number jumped from 41 to 172, most requests got cancelled, and the list had gaps.

● Before — checks loading (state)



One fast flick → 6 requests, gaps in the list

● After — checks isFetchingRef (ref)



One flick → exactly 1 request

Why it happened

Each render makes a new scroll function, and that function remembers the value of loading from the moment it was made — like a photo.

When we call `setPage`, React doesn't update right away; there's a tiny gap before the next render.

During a fast scroll, the observer fired again inside that gap, using the old function that still "saw" `loading = false`. So it asked for another page, and another.

A ref is different: it's one shared box. Every function, old or new, looks inside the same box, so it always sees the latest value.

In one line: the scroll function was reading an old copy of loading, so it didn't know a request was already running.

The code

Before — checks loading (state)

```
const { loading } = useFetch(url);

useInfiniteScroll(ref, () => {
  // this is an old copy of loading
  if (hasMoreRef.current && !loading) {
    setPage((p) => p + 1);
  }
});
```

After — checks isFetchingRef (ref)

```
const { isFetchingRef } = useFetch(url);  
// useFetch sets this to true the moment a request starts  
  
const onIntersect = useCallback(() => {  
  if (hasMoreRef.current && !isFetchingRef.current) {  
    setPage((p) => p + 1);  
  }  
}, []);
```

How to avoid it next time

- If a value is checked inside something that can fire between renders (observer, timer, event listener), use a ref.
- If a value needs to show on screen, use state. If you need both, keep one of each.

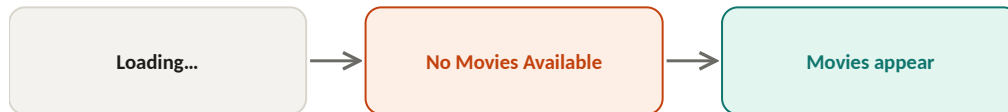
Bug #6

"No Movies Available" flashes before the movies appear

What you saw

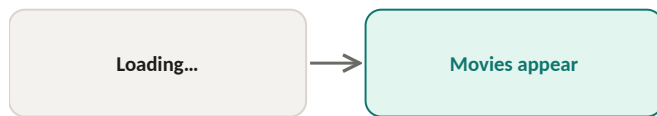
On every page load (and when coming back from a movie), "No Movies Available" flashed for a split second before the movies showed up.

● Before — finally always updates loading



The cancelled request sets loading = false while the real one is still running.

● After — only if the request wasn't cancelled



The cancelled request stays quiet. Only the real request finishes loading.

Why it happened

In development, React StrictMode starts the fetch twice. The first one gets cancelled straight away.

In useFetch, the finally block always runs — even for the cancelled request. So the cancelled request set loading to false while the real request was still loading.

For a moment the screen had loading = false and no movies, so the empty message showed.

The same thing can happen for real users when a search word changes quickly, because the old request gets cancelled too.

In one line: the cancelled request said "done loading" while the real one was still loading.

The code

Before — finally always updates loading

```
} finally {  
  // runs even for the cancelled request  
  setLoading(false);  
  isFetchingRef.current = false;  
}
```

After — only if the request wasn't cancelled

```
} finally {  
  // the cancelled request stays quiet;  
  // the real request updates loading when it finishes  
  if (!controller.signal.aborted) {  
    setLoading(false);  
    isFetchingRef.current = false;  
  }  
}
```

How to avoid it next time

- In finally, only update state if the request wasn't cancelled (check `controller.signal.aborted`).
- Decide "empty" from the response itself (`data.results.length === 0`), not from a list that updates one step later.
- Keep StrictMode on — it showed this bug early, before real users hit it.

Bug #10

Movie details page is completely blank

What you saw

Clicking a movie opened a white page. No errors in the console, the network request returned 200, and logging showed the movie data had arrived.

Before — no return at the end



Network 200 • data arrived • no errors

After — the full JSX is back



Poster, title, overview, related movies

Why it happened

While editing, a placeholder comment — `// ...rest is unchanged (JSX)` — was pasted into the file in place of the real code.

So the component had no return with JSX anymore. A function with no return gives undefined.

React shows nothing for undefined and doesn't complain. Everything worked — there was just nothing to draw.

In one line: the JSX was replaced by a comment, so the component returned nothing.

The code

Before — no return at the end

```
if (!movieDetails) return <p>Movie not found</p>;

console.log(movieDetails);
// ...rest is unchanged (JSX)
} // ← nothing returned
```

After — the full JSX is back

```
export default function MovieDetails(): ReactElement {
  ...
  if (!movieDetails) return <p>Movie not found</p>;

  const { title, overview, ... } = movieDetails;

  return (
    <div className="details"> ... </div>
  );
}
```

How to avoid it next time

- Give components a return type (`: ReactElement`) so TypeScript complains if a return is missing.
- Turn on `noImplicitReturns` in `tsconfig`.
- When pasting code that contains `"..."` placeholders, check the file didn't lose a big chunk of lines.

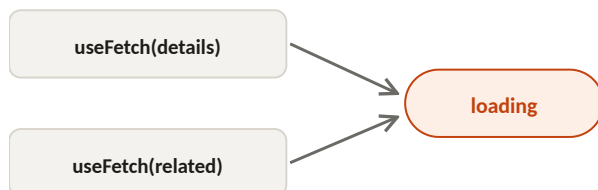
Bug #11

Two useFetch calls use the same names

What you saw

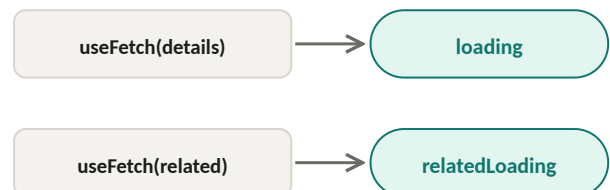
The app stopped with "Identifier 'loading' has already been declared". When only data was renamed, the details page waited for the related movies before showing anything.

● Before — same names twice



Same name twice → error, or one page part waits for the other

● After — every value renamed



Each part has its own name

Why it happened

Each useFetch call returns data, loading and error.

Calling it twice in one component gives two sets with the same names — JavaScript doesn't allow the same name twice.

When only some names were changed, both parts of the page ended up sharing one loading flag, so the main movie waited for the related movies.

In one line: two useFetch calls, same names → they clash.

The code

Before — same names twice

```
const { data, loading, error } = useFetch(detailsUrl);
const { data, loading, error } = useFetch(relatedUrl);
//      ↑ already declared
```

After — every value renamed

```
const { data: movieDetails, loading, error } =
  useFetch(detailsUrl);

const {
  data: relatedMovies,
  loading: relatedLoading,
  error: relatedError,
} = useFetch(relatedUrl);
```

How to avoid it next time

- When you call a hook twice, rename every value it returns, not just data.

- Let each part of the page use its own loading and error, so a slow part doesn't block a fast one.

Bug #13

Search and popular movies load pages together

What you saw

Searching "batman" and scrolling fired both `popular?page=...` and `search?page=...` every time, climbing together. Half the requests were never shown on screen.

Before — one page for both



One scroll → two requests

After — each list has its own



One scroll → one request

Why it happened

Both URLs used the same page state.

Increasing page changed both URLs, so both lists fetched a new page — even though only one list was on screen.

Clearing the search also meant popular movies jumped to whatever page the search had reached.

In one line: one page number was shared by two lists.

The code

Before — one page for both

```
const [page, setPage] = useState(1);

const url = `.../movie/popular?page=${page}`;
const searchUrl = `.../search/movie?query=${q}&page=${page}`;
// setPage → BOTH requests fire
```

After — each list has its own

```
const [page, setPage] = useState(1);
const [searchPage, setSearchPage] = useState(1);

const url = `.../movie/popular?page=${page}`;
const searchUrl = `.../search/movie?query=${q}&page=${searchPage}`;

// handleIntersect picks ONE:
if (debouncedSearch) setSearchPage((p) => p + 1);
else setPage((p) => p + 1);
```

How to avoid it next time

- Each list gets its own page, its own list of movies, its own hasMore, and its own fetching flag.
- In the scroll handler, decide which list to load based on the current mode.

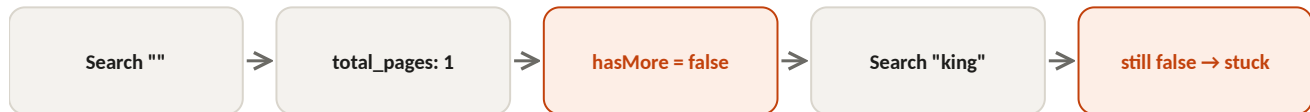
Bug #14

A new search can't load more pages

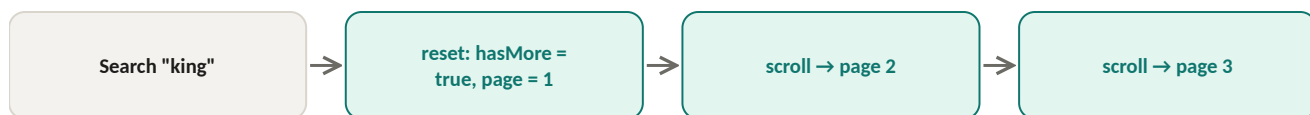
What you saw

After an empty search, typing "king" showed 20 results, but scrolling to the bottom did nothing. Logs showed `hasMoreSearchRef`: false.

● Before — only ever set to false



● After — reset when the search word changes



Why it happened

An empty search came back with `total_pages: 1`, so `hasMoreSearchRef` was set to false — correct for that search.

But nothing ever set it back to true. A ref keeps its value for as long as the component is on the page.

So every later search ("king", "batman") still thought there were no more pages.

In one line: "no more pages" from an old search was never reset for the new one.

The code

Before — only ever set to false

```
if (searchResults.page >= searchResults.total_pages) {
  hasMoreSearchRef.current = false; // never set back
}
```

After — reset when the search word changes

```
// new search word -> start fresh
useEffect(() => {
  setSearchPage(1);
  setSearchMovies([]);
  setSearchHasMore(true);
  hasMoreSearchRef.current = true;
}, [debouncedSearch]);
```

How to avoid it next time

- If you set something to false for one search, reset it when the search changes.
- Don't fetch at all when the search box is empty — it only returns an empty result that can confuse other logic.

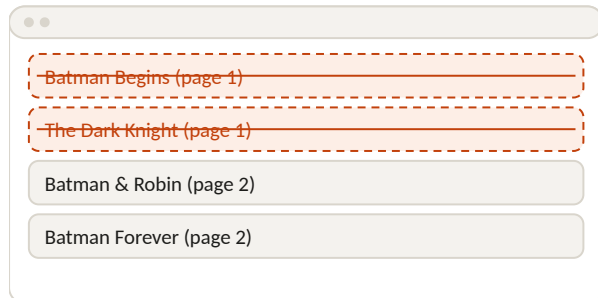
Bug #15

New page wipes out the old one

What you saw

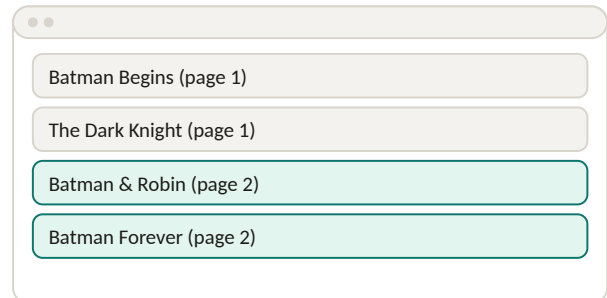
In search, loading page 2 made page 1's movies disappear. The list always showed 20 movies, and the scrollbar jumped around.

● Before — shows only the latest page



Page 1 removed → always 20 movies

● After — adds each page to the list



Page 2 added under page 1 → 40 movies

Why it happened

useFetch replaces data with each new response — it doesn't keep old pages.

Popular movies worked because we copied each page into our own list (popularMovies).

Search didn't have that list; it showed searchResults.results directly, which is only ever the latest page.

Because the old rows were removed, the browser also lost track of where you were reading, so the scroll position jumped.

In one line: search showed only the newest page instead of adding it to the pages before.

The code

Before — shows only the latest page

```
const { data } = useFetch(searchUrl);

// only the latest page
<MovieList movies={data?.results ?? []} />
```

After — adds each page to the list

```
const [items, setItems] = useState<Movie[]>([]);

useEffect(() => {
  if (!data) return;
  setItems((prev) => [...prev, ...data.results]);
}, [data]);

// start fresh when the search word changes
useEffect(() => { setItems([]); }, [query]);

<MovieList movies={items} />
```

How to avoid it next time

- For any list that loads more than one page, keep your own list and add each page to it.
- Use `key={movie.id}`, not `key={index}`.
- Clear the list when the search word changes.

Bug #16

Search loads once, then stops

What you saw

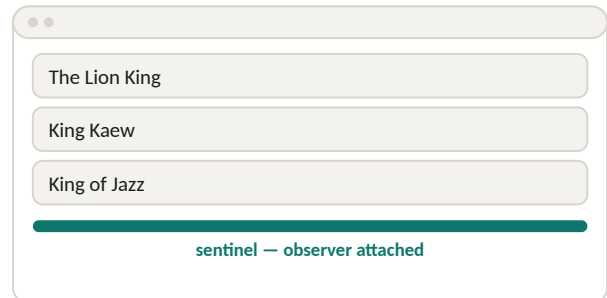
Search results showed up, but scrolling to the bottom did nothing — no new movies, no request — even though there were 256 pages available.

Before — plain useRef



Scroll to the bottom → nothing happens

After — function ref + state



div appears → setNode → observer starts

Why it happened

The sentinel only appears once search results arrive. But the code that starts the observer ran earlier, right when the page loaded.

At that moment the sentinel didn't exist yet, so the code gave up.

When the sentinel appeared later, React quietly put it in the ref — but changing a ref doesn't tell React to run anything again. So the observer never started, and nobody was watching.

The fix: pass a function as the ref. React calls that function the moment the div appears. Inside it we save the div into state with `setNode`. State changes do make React run things again, so the observer finally starts.

In one line: the observer only got one chance to start, and the sentinel wasn't there yet.

The code

Before — plain useRef

```
const sentinelRef = useRef<HTMLDivElement>(null);

useEffect(() => {
  // runs once, when the sentinel doesn't exist yet
  if (sentinelRef.current === null) return;
  observer.observe(sentinelRef.current);
  // the ref box never changes, so this never runs again
}, [sentinelRef, onIntersect]);
```

After — function ref + state

```
const [node, setNode] = useState<HTMLDivElement | null>(null);

// React calls this when the div appears
const sentinelRef = useCallback(
  (el: HTMLDivElement | null) => setNode(el), []);

useEffect(() => {
  if (node === null) return;
  observer.observe(node);
  return () => observer.disconnect();
  // node changes from null to the div, so this runs again
}, [node, onIntersect]);
```

How to avoid it next time

- If an element appears later (inside a condition), use a function ref + state, not a plain useRef.
- Let the hook create and return the ref, so the caller can't wire it up wrong.
- When you switch to the new hook, delete the old useRef line.

Observer keeps restarting on every render

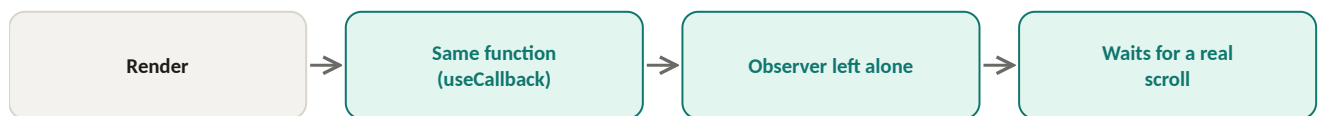
What you saw

During search, pages kept loading one after another with the mouse completely still. The log flipped true, false, true, false over and over.

Before — function written inline



After — wrapped in useCallback



Why it happened

Writing `() => {...}` directly inside the component makes a brand new function on every render.

The scroll hook sees "new function" and restarts the observer.

Every time the observer restarts, it checks right away: "is the sentinel visible?" If yes, it loads a page.

The new page causes a render, the render makes a new function, the observer restarts, checks again... and it loops on its own.

In one line: a new function every render → observer restarts every render → checks right away every time → loads without scrolling.

The code

Before — function written inline

```
// a new function on every render
useInfiniteScroll(ref, () => {
  if (hasMoreRef.current && !fetchingRef.current) {
    setPage((p) => p + 1);
  }
});
```

After — wrapped in useCallback

```
const onIntersect = useCallback(() => {  
  if (hasMoreRef.current && !fetchingRef.current) {  
    setPage((p) => p + 1);  
  }  
  // same function every render, unless the search changes  
}, [debouncedSearch]);  
  
const ref = useInfiniteScroll(onIntersect);
```

How to avoid it next time

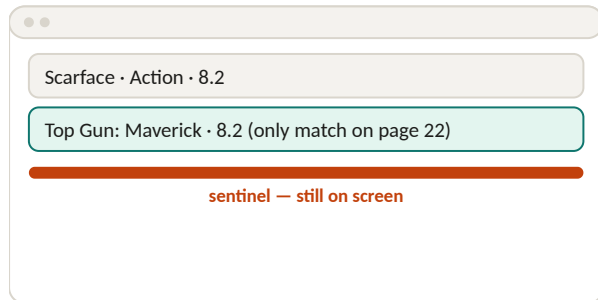
- If you pass a function to a hook that uses it inside useEffect, wrap it in useCallback.
- Refs and setState functions don't need to go in the dependency list.

Loading stops when filters are on

What you saw

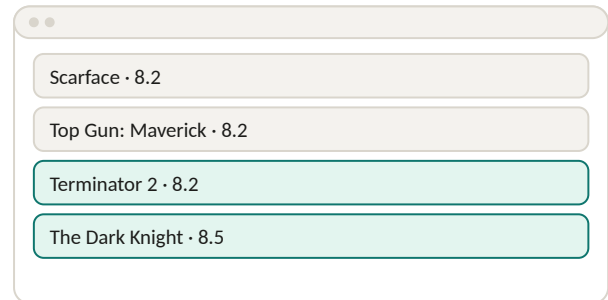
With "Action" and rating "8" selected, scrolling to the bottom loaded one page and then got stuck. Scrolling up and back down loaded just one more movie, then stuck again.

● Before — waits for a hidden → visible change



Visible before, visible after → no signal

● After — look again after each page, with a limit



Each page → look again → until the screen fills

Why it happened

When filters are on, each new page adds very few movies to the screen — sometimes just one, sometimes none.

Because so little gets added, the list doesn't grow tall enough to push the sentinel off the screen.

The observer only fires when the sentinel goes from hidden to visible. It was already visible, and it's still visible — so it never fires again.

Result: loading stops, even though TMDB has hundreds more pages that could contain matching movies.

In one line: too few filtered movies get added → sentinel never leaves the screen → observer never gets a "new" signal → stuck.

Why scrolling up and down "fixes" it once

Scrolling up hides the sentinel. Scrolling back down shows it again. That hidden → visible change is the signal the observer needs — so it loads one more page, which adds maybe one movie, and you're stuck again.

How it was confirmed

In the Elements panel, hovering the height: 10px div showed it highlighted on screen, right under the last movie — not below the bottom of the screen.

Why popularMovies still grows by 20 with filters on

Loading and filtering are two separate steps. TMDB sends 20 movies and all 20 are saved into popularMovies — filters aren't used there. Then filterMovies() makes a separate, smaller list to show. Filters never remove anything from popularMovies.

The loop, watching the sentinel

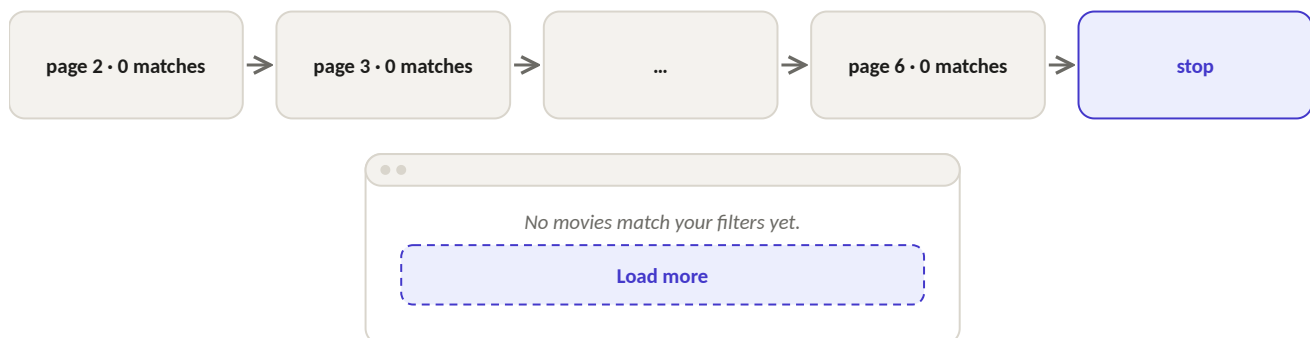
1. **You scroll to the bottom.** The sentinel comes onto the screen. Observer: "it just appeared!" → load the next page.
2. **A page loads, 1 movie passes the filter.** The sentinel moves down a little but is still on screen.
3. **popularMovies grew by 20.** loadedCount changed → the observer restarts and looks again: "on screen?" → yes → load the next page.
4. **A page loads, 0 movies pass.** The sentinel doesn't move at all. The observer restarts again → still on screen → load the next page.
5. **Enough matches finally arrive.** The sentinel is pushed below the screen. Observer: "on screen?" → no → stop, and wait for you to scroll.

The key point: the sentinel decides when to stop. Loading keeps going while it is still on screen, and stops once enough movies pile up above it to push it off the bottom.

How it was fixed

1. **Look again after each page.** Pass loadedCount (the length of the list that's loading) into useInfiniteScroll and add it to the effect's dependency list. Every time a page lands, loadedCount grows by 20, the observer restarts once, and checks: "is the sentinel still on screen?" If yes, it loads another page.
2. **Why not the filtered list's length.** A page with zero matches doesn't change the filtered length, so the observer wouldn't look again. The full list always grows by 20.
3. **A limit of 5 pages in a row.** With a rare filter (like War + 9) the screen may never fill, so it could load all 1,000+ pages. A counter (a ref) stops after 5 pages in a row and shows a Load more button.
4. **Reset the counter when the screen fills.** The observer now also reports when the sentinel goes OFF screen (onLeave). That means enough movies arrived, so the counter goes back to 0. Normal browsing never reaches 5.
5. **Messages.** "No movies match your filters yet." when nothing matched, or "Only 3 matches so far." — plus a Load more button. Apply and Clear filters also reset the counter.

Rare filter (War + 9): the limit stops it after 5 pages



The code

Before — waits for a hidden → visible change

```
useEffect(() => {  
  ...  
}, [node, onIntersect]);  
// only restarts when the sentinel appears  
// or the search changes
```

After — look again after each page, with a limit

```
// useInfiniteScroll  
useEffect(() => {  
  const observer = new IntersectionObserver([entry] => {  
    if (entry.isIntersecting) onIntersect();  
    else onLeave?.();           // screen filled up  
  });  
  observer.observe(node);  
  return () => observer.disconnect();  
}, [node, onIntersect, onLeave, loadedCount]);  
  
// Home.tsx  
const loadedCount = activeMovies.length;  
if (autoLoadCountRef.current >= 5) {  
  setShowLoadMore(true);  
  return;  
}  
autoLoadCountRef.current += 1;
```

How to avoid it next time

- If you filter on the client, each page may add very few items — plan for the sentinel staying on screen.
- After each page loads, have the observer look again, and cap how many pages it can load by itself.
- Or let the API filter for you (TMDB /discover/movie), so every page comes back full.

Summary

Things to remember

- The observer checks once right away when it starts watching, and after that only fires when something goes from hidden to visible or back (#4, #17, #20).
- State is a copy per render; a ref is one shared box with the latest value (#5, #16).
- The dependency list decides when an effect runs again — too few and it never re-runs (#16), a new function every render and it re-runs constantly (#17).
- useFetch replaces data, so a list that loads many pages needs its own list that grows (#15).
- Anything set for one search must be reset when the search changes (#14), and each list needs its own page number (#13).
- In finally, only update state if the request wasn't cancelled (#6).